Lab Today (week 3)

- **Lab Lecture**
  - Solving Issues with TCP Receive (MSG_WAITALL flag)
  - What happens when a TCP connection closes
  - TCP main server socket vs. worker sockets
  - Managing TCP Connections (poll())

Issues with TCP Receive

client – server

TCP – send()s/recv()s a stream of bytes

↳ called by server

TCP just a stream of bytes
– similar to a disk write.

- TCP communications is data **stream** oriented

- Send **two** messages: **"Networks", "Rule"**

send()
send()
send()

recv()

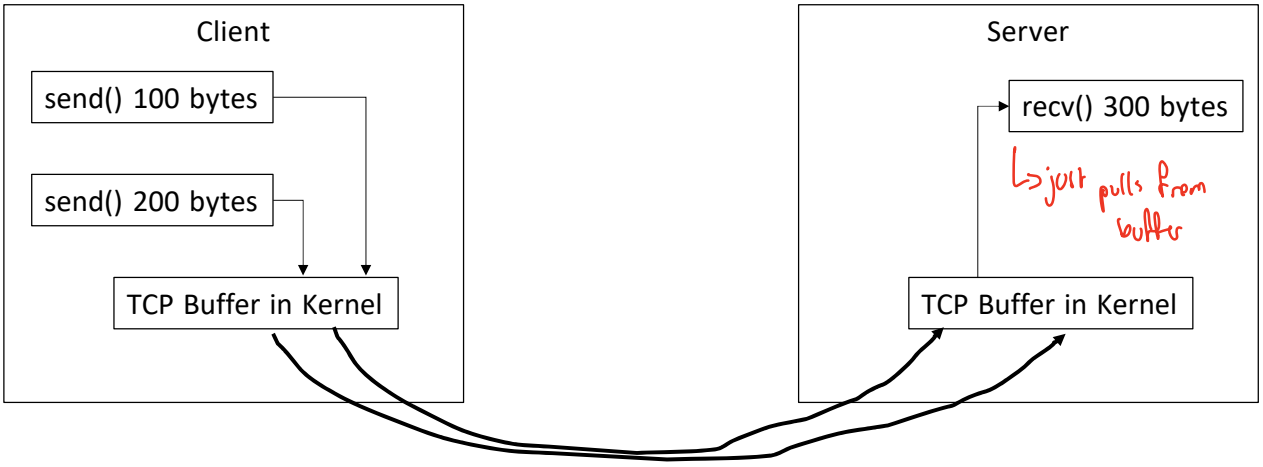
You may recv():

	recv() call #1	recv() call #2	What happened
1	Networks	Rule	recv() as sent
2	Networks Ru	Le	Second send() split
3	Net	Works rule	First send() split
4	Networks Rule	(call blocks)	Both send()s combined

Problem: TCP doesn't respect our messages boundaries.

(not 1 to 1 send() to recv())

↳ doesn't guarantee it won't break it up



MSG_WAITALL

Flag we use in TCP recv

recv (socket , buffer, len, MSG_WAITALL)

flag

MSG_WAITALL = block and recv() len # of bytes

only return when it gets len() # of bytes

We read 2 into a short,

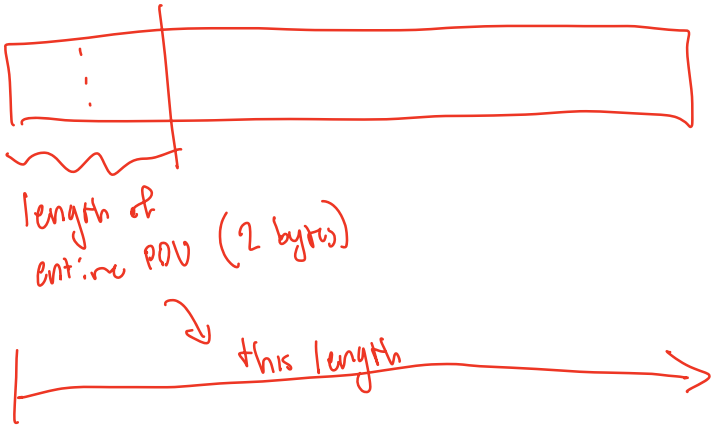
convert to host order ntohs

length is entire PDU including itself.

Recv()ing our PDU in TCP

Our messages have a defined boundary we need to respect

To get this to work: create application level PDU



now we can recieve length, tells us how many more bytes to get to the end of that PDU

recv() → length of PDU
recv() → rest of PDU

Note - There are other ways to solve this problem

Lab 3 Program: 1-send(), 2-recv()s

The sendPDU(), recvPDU() worker functions

- int **sendPDU**(int clientSocket, uint8_t * dataBuffer, int lengthOfData);
- int **recvPDU**(int socketNumber, uint8_t * dataBuffer, int bufferSize);

sendPDU()

- creates/adds the PDU length to the PDU
- does one send()

data buffer + len of data

memcopy data into new buffer, add 2 bytes for length of data

- send entire buffer (socket, PDU, PDULen); network order

recvPDU()

two recv()

-1. recv (sk, buffer, 2, MSG_WAITALL)

PDU len in network order
- convert to host
- subtract 2

-2. recv (sk, data_buffer, buffer_len(-2), MSG_WAITALL)

TCP Closed Connections

Detecting a closed socket when recv()ing

- How do we know if the other end closed the connection?

- Check the return value from recv()

  - Two cases:
    - Returns 0 → other side closed it
      - can't send/recv 0 bytes ← everyone does this
    - Returns -1 and errno = ECONNRESET
      - → connection is closed

send() and SIGPIPE (broken pipe)

→ writing to closed connection

→ do not catch error, fix program

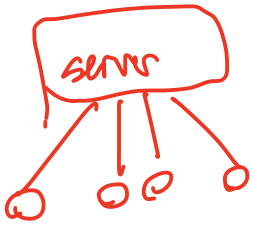
- How do you get this?
  - send() on closed connection

- In Program #2 what should you do?

logic error
→ fix your bug

TCP's Main Server Socket vs. Worker sockets

main server talks to multiple clients



not multiple processes / threads

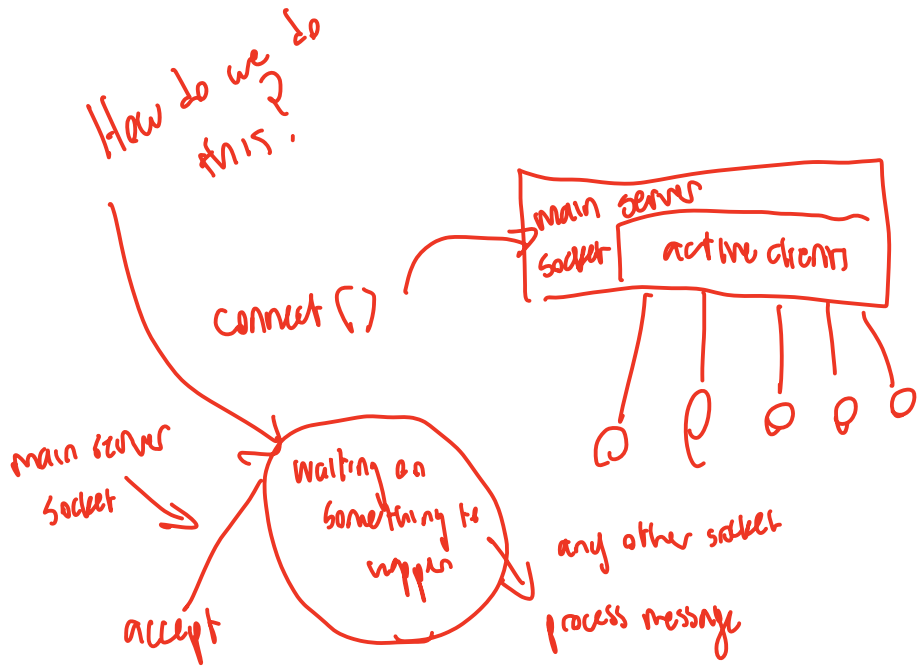
TCP Connection

Client		Server
sk =socket()		s1=socket()
		bind(s1,…)
(note: need server port #)		getsockname(….)
		printf(… port …)
		listen(s1,…)
connect(sk,...)	← →	s2 = accept(s1,…)
		(accept blocks waiting on a connect())
		(Could create another thread here)
		recv(s2,…)
send(sk…)	→	
		send(s2…)
recv(sk…)	←	
		close(s2)
close(sk)	← →	
		(when server is shutting down)
		close(s1)

could accept, keep getting new children, new socket #

send, recv() on that socket #

On the **Server**: TCP <u>main server socket</u> and <u>worker sockets</u>



TCP Connection

<u>Client</u>		<u>Server</u>
sk =socket()		mainServerSocket = socket()
(note: need server port #)		bind(mainServerSocket,…)
		getsockname(….)
		printf(… port …)
		listen(mainServerSocket,…)
connect(sk,...)	↔	worker1 = accept(mainServerSocket,…)
		(accept blocks waiting on a connect())
		(Could create another thread here)
		recv(worker1,…)
send(sk…)	→	
		send(worker1 …)
recv(sk…)	←	
		close(worker1)
close(sk)	↔	
		(when server is shutting down)
		close(mainServerSocket)

Managing Connections

Using poll() to check if a socket is ready for reading

Problem – both accept() and recv() are blocking calls

blocking

accept() recv()

poll() watches

blocking

process user input

process incoming message

STDIN Fileno as something to watch

Use poll to watch sockets

Two types of problems we need to solve

- Server working with many clients at the same time (program #2)
- Waiting to recv() something for a specific period of time (program #3 – more later)

Server with multiple clients

Probing our sockets (looking at them) using poll()

- Using the poll() call to "probe" our sockets (look at)

  does not receive anything, just looks

- We can probe (look at) for a configurable duration (time)
  - Block indefinitely until a socket is ready for read 1
  - For a specific amount of time but return immediately when something is ready for read 2
  - Return immediately and just tell me if anything is available for read 3

What might we want to watch (probe/monitor)

poll() works on file descriptors (older select() call)

- add to the set each socket
- call poll() on set of sockets
- When it returns, check which socket caused poll to return

How we do it

- Create a set of sockets (file descriptors) to watch
- Call poll() on that set of file descriptors with a given it a time value
  - Call has a set of file descriptors and a time value.
- On return, process each socket in the returned set that is ready for recv()
  - Must look to see which ones are ready
  - Must do the recv()/accept() on that socket

anything else

main server socket

include in poll set

poll() system call

- int poll(struct pollfd *fds, nfds_t nfds, int timeout);

file descriptors

# of fds

timeout value

-1 = block forever

poll() given

- not multithreaded safe

pollLib()

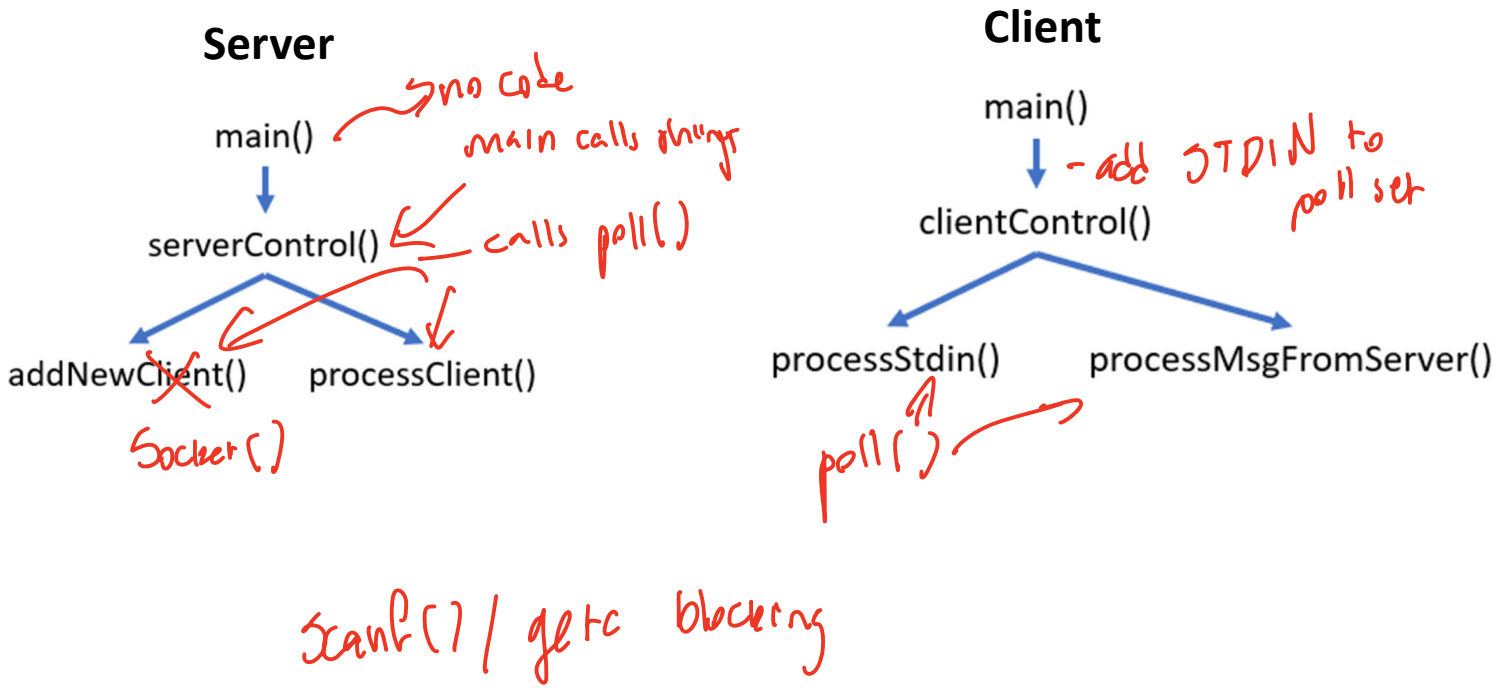
init,
add sockets to it,

- void setupPollSet();
  - Initializes the library
- void addToPollSet(int socketNumber);
- void removeFromPollSet(int socketNumber);
- int pollCall(int timeInMilliSeconds);
  - Returns socket ready for read
  - Returns -1 if nothing is ready to read
  - If time = -1 then block forever

Determine if a Connection closes with poll()

- poll() returns when a socket is closed by the other end

- call poll()
  - returns OK
  - recv 0 bytes



scanf() / getc blocking